Assignment: Match-3 Adventure Game & Level Progression Platform

Overview

Build a modern **Match-3 Adventure Game** using **Svelte 5, SvelteKit, and StencilJS**.

The application should provide an engaging tile-matching game experience where players complete progressively challenging levels by matching game pieces, achieving objectives, earning points, and managing limited moves.

The game should be inspired by the **match-3 genre** but must have its own visual identity, game mechanics, level design, and user experience. It should **not be a direct copy of Candy Crush or any existing game**.

In addition to developing the game application, you are required to design and develop a **reusable StencilJS component library**, publish it as an npm package, and consume the **published npm package** within the SvelteKit application.

The solution should demonstrate your understanding of:

- Svelte 5 and SvelteKit
- Component-based architecture
- State management
- Game logic and algorithms
- Responsive/mobile-first UI development
- Animations and transitions
- Routing
- Web Components
- StencilJS
- API/local data integration
- Custom events
- Component properties
- Slots
- npm package development and publishing
- Frontend architecture

You are encouraged to make independent design and implementation decisions.

Game Concept

Create an original match-3 game with a theme of your choice.

For example:

"Mystic Gems" — Players travel through different worlds, completing match-3 puzzle levels by matching magical gems and completing level-specific objectives.

You may choose a completely different theme such as:

- Space exploration
- Ocean adventure
- Ancient civilizations
- Fantasy kingdom
- Robots
- Jungle adventure
- Food/cooking
- Futuristic cyber world

The theme, visual design, characters, game pieces, animations, and level progression should be original.

Functional Requirements

1. Home / Game Dashboard

Create a visually appealing home screen containing:

- Game logo/title
- Player profile/avatar
- Player score or accumulated stars
- Current level
- Play button
- Level selection
- Lives/energy indicator
- Settings button
- Favorites or achievements section where applicable

The dashboard should provide clear navigation to the main game features.

---

2. Level Selection

Create a level-selection screen where users can browse available levels.

Each level should display:

- Level number
- Locked/unlocked state
- Completion status
- Stars earned
- Best score
- Level difficulty

Example:

Level 1 ⭐⭐⭐

Level 2 ⭐⭐

Level 3 ⭐

Level 4 🔒

Level 5 🔒

Levels should progressively increase in difficulty.

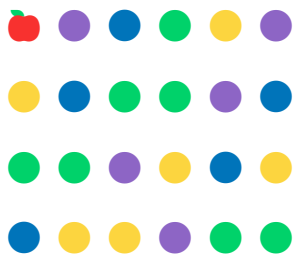
The application should prevent users from playing locked levels unless the required previous level has been completed.

---

3. Match-3 Game Board

Implement the core match-3 gameplay.

The game board should contain a grid of tiles, for example:



The player should be able to:

- Select a tile
- Select an adjacent tile
- Swap tiles
- Detect valid matches
- Remove matched tiles
- Apply gravity
- Generate new tiles
- Continue detecting chain reactions

A valid match should consist of at least **three matching adjacent tiles** horizontally or vertically.

Diagonal matching is optional.

---

4. Game Mechanics

Implement the complete match-3 gameplay loop.

Basic flow

Player selects tile

↓

Player selects adjacent tile

↓

Swap tiles

↓

Check for matches

↓

Match found?

↙ ↘

YES NO

↓ ↓

Remove Revert

tiles swap

↓

Apply gravity

↓

Generate new tiles

↓

Check chain reaction

↓

Update score

↓

Check level objective

The implementation should handle:

- Tile swapping
- Invalid swaps
- Match detection
- Multiple simultaneous matches
- Cascading matches
- Tile removal
- Tile falling
- New tile generation
- Score calculation
- Move counting

---

5. Special Tiles

Implement at least **two special tile mechanics**.

Examples:

Line Blast

Matching four tiles creates a special tile that can clear an entire row or column.

Bomb

Matching five tiles creates a bomb that clears tiles within a specific radius.

Color Blast

Matching five tiles creates a special tile capable of clearing all tiles of a selected type.

You may implement different mechanics if they provide equivalent complexity.

The special tiles should have distinct visual representations and effects.

---

6. Level Objectives

Each level should have a specific objective.

Examples:

Score Objective

Reach 10,000 points

Tile Collection

Collect:

🔵 Blue Gems: 20

🟡 Yellow Gems: 15

Obstacle Destruction

Destroy 10 ice blocks

Limited Moves

Complete the objective within 25 moves

Different levels should be able to have different objectives.

---

7. Obstacles

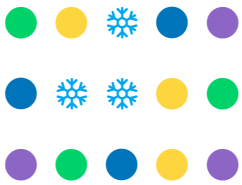
Implement at least **one type of obstacle**.

Examples:

- Ice blocks
- Locked tiles
- Stone blocks
- Chains
- Bomb tiles
- Jelly
- Walls

Obstacles should affect gameplay rather than being purely decorative.

For example:



Matching adjacent tiles could progressively remove an ice block.

---

8. Scoring System

Implement a scoring system.

The score should consider factors such as:

- Number of tiles matched
- Special tiles
- Cascading combinations
- Combo multiplier
- Remaining moves

For example:

3 Match → +30

4 Match → +60

5 Match → +100

Combo → ×2

Special Tile → Bonus

The exact scoring algorithm is left to the candidate.

---

9. Combo / Cascade System

Implement a combo or cascade mechanism.

For example:

Match 1

↓

Tiles fall

↓

Automatic Match 2

↓

Combo ×2

↓

Automatic Match 3

↓

Combo ×3

The UI should provide visual feedback when a combo occurs.

Examples:

NICE!

GREAT!

COMBO ×2!

AMAZING!

---

10. Game Timer / Move System

The game should use a **limited number of moves**.

For example:

LEVEL 12

Moves: 18 / 30

Score: 8,450

Target: 12,000

The player should lose the level when:

- The objective has not been completed
- Available moves reach zero

The game should end immediately when the win/loss condition is reached.

---

11. Level Completion

When the player completes a level, display a result screen.

The result should include:

- Final score
- Stars earned
- Remaining moves
- Bonus score
- Level completion status
- Next level button
- Replay button
- Return to level selection

Example:

LEVEL COMPLETE!

SCORE

15,450

+2,000 Bonus

[NEXT LEVEL]

[REPLAY LEVEL]

---

12. Game Over

Create a game-over experience.

Display:

- Final score
- Objective progress
- Remaining/used moves
- Retry option
- Exit option

Example:

LEVEL FAILED

Score: 8,450

Target: 12,000

❌ Objective not completed

[TRY AGAIN]

[LEVELS]

---

13. Lives / Energy System

Implement a basic lives system.

For example:

❤️ ❤️ ❤️ ❤️ ❤️

A failed level can consume one life.

When no lives remain, the user should not be able to start another level until the lives are restored.

For the assignment, lives can be restored using:

- A countdown timer
- A mock "Restore Life" action
- Or a simple automatic regeneration mechanism

No real-money payment system is required.

---

14. Player Progress

Persist player progress.

The application should remember:

- Current level
- Unlocked levels
- Completed levels
- Stars
- Best scores
- Lives
- Achievements where applicable

You may use:

- LocalStorage
- IndexedDB
- A lightweight backend/API
- Mock persistence

A backend is **not mandatory**, but the architecture should allow one to be introduced later.

---

15. Achievements

Implement an achievements section.

Examples:

🏆 First Victory

Complete your first level.

🔥 Combo Master

Achieve a 5× combo.

💎 Gem Collector

Match 500 tiles.

⭐ Perfect Player

Earn 3 stars on 5 levels.

Achievements should be unlocked based on actual gameplay events.

---

16. Game Settings

Provide a settings screen containing options such as:

- Sound on/off
- Music on/off
- Animation on/off
- Reset progress
- About the game

The settings should persist between sessions.

---

17. Responsive / Mobile-First Design

The game should primarily be designed for **mobile devices**.

The application must work properly across:

- Mobile
- Tablet
- Desktop

The game board should adapt to different screen sizes without breaking the gameplay.

The UI should support touch interactions.

Desktop mouse interaction should also be supported.

---

18. Navigation & Routing

Use **SvelteKit routing**.

At minimum, implement routes similar to:

/

/levels

/game/[levelId]

/results/[levelId]

/achievements

/settings

You may introduce additional routes if required.

The game state should be handled appropriately when navigating between routes.

---

19. State Management

Use Svelte 5's modern state management capabilities appropriately.

The application should maintain state for:

- Current player
- Current level
- Game board

- Selected tile
- Score
- Moves
- Lives
- Objectives
- Special tiles
- Game status
- Level progress
- Settings
- Achievements

Avoid unnecessarily duplicating state across components.

The game logic should be separated from UI components wherever practical.

---

20. Game Engine / Architecture

Separate the core game logic from the UI.

For example:

src/

├── lib/

│ ├── game/

│ │ ├── board.ts

│ │ ├── match-detector.ts

│ │ ├── tile-manager.ts

│ │ ├── scoring.ts

│ │ ├── level-manager.ts

│ │ └── game-engine.ts

│ │

│ ├── stores/

```
│   ├── api/
│   └── utils/
│
├── routes/
│   ├── +page.svelte
│   ├── levels/
│   ├── game/
│   ├── achievements/
│   └── settings/
```

The exact architecture is up to the candidate.

StenciJS Component Library

In addition to the SvelteKit application, create a reusable **StenciJS Web Component library**.

The component library should contain reusable components that could theoretically be used by other applications.

Required Stencil Components

Implement at least **5 reusable Web Components**.

Suggested components:

1. Game Tile

```
<game-tile
  type="blue"
  selected="true"
  special="bomb">
</game-tile>
```

Responsibilities:

- Display tile

- Selected state
- Special tile state
- Animation state
- Click/tap interaction

---

2. Game Board

<game-board

.boardData={board}

moves="20">

</game-board>

Responsibilities:

- Render the game board
- Handle tile interactions
- Emit tile selection/swap events

---

3. Score Display

<game-score

score="12500"

target="15000">

</game-score>

---

4. Life Counter

<life-counter

lives="3"

max-lives="5">

</life-counter>

---

5. Level Card

```
<level-card
 level="12"
 stars="3"
 score="15200"
 locked="false">
</level-card>
```

Additional Stencil Components

You may implement additional components such as:

- Game Button
- Progress Bar
- Achievement Card
- Game Modal
- Combo Indicator
- Star Rating
- Timer
- Game Header
- Toast Notification
- Power-Up Button

A polished component library with more reusable components will be considered positively.

Stencil → Svelte Integration

The SvelteKit application **must consume the published StencilJS npm package**.

Do **not** import Stencil components directly from the component library source code.

The flow should be:

StencilJS Component Library

↓

npm publish

↓

npm package

↓

SvelteKit

↓

Game Application

For example:

npm install @your-scope/game-ui

Component Properties

Demonstrate passing data from Svelte to Stencil components using properties.

Example:

```
<game-score
  score={gameState.score}
  target={currentLevel.targetScore}
/>
```

The Stencil component should react appropriately when its properties change.

Custom Events

Stencil components must emit custom events.

For example:

game-tile-selected

tile-swapped

game-completed

game-over

level-selected

SvelteKit should listen to these events and update application state.

Example:

```
<game-board
  boardData={board}
  on:tileSelected={handleTileSelected}
/>
```

Use the appropriate Svelte 5 event/listener approach based on your implementation.

Slots

Demonstrate the use of **slots** in at least one Stencil component.

For example:

```
<game-modal>
  <span slot="title">Level Complete!</span>

  <div slot="content">
    You earned 3 stars.
  </div>

  <button slot="actions">
    Continue
  </button>
</game-modal>
```

The slot implementation should provide a meaningful reusable use case rather than being added only to satisfy the requirement.

Animation & User Experience

The game should feel polished.

Implement appropriate animations for:

- Tile selection

- Tile swapping
- Matched tiles disappearing
- Tiles falling
- New tiles appearing
- Special tile activation
- Combos
- Score updates
- Level completion
- Game over

Animations should not interfere with gameplay.

Avoid excessive animations that make the game difficult to play.

API / Data Integration

The application should have a clear strategy for level/game configuration.

You may use:

- Local JSON data
- A mock API
- A public API
- A lightweight custom API

For example, level configuration could be:

```
{
  "id": 12,
  "difficulty": "medium",
  "boardSize": 8,
  "moves": 25,
  "objective": {
    "type": "collect",
    "tile": "blue",
```

```
  "count": 30
 }
}
```


The candidate should avoid hardcoding level logic directly into UI components.

Error & Edge Case Handling

The application should gracefully handle scenarios such as:

- Invalid level ID
- Locked level
- No available moves
- Invalid tile swap
- Rapid repeated clicks
- Level data unavailable
- Unexpected game state
- Resetting game progress
- Browser refresh during a game

Code Quality Requirements

The implementation should demonstrate:

- Clean component structure
- Reusable components
- Meaningful naming
- TypeScript where appropriate
- Separation of concerns
- Minimal duplication
- Proper state management
- Maintainable game logic
- Responsive design

- Accessibility considerations
- Error handling

Avoid putting the entire game implementation inside a single .svelte file.

Testing

Implement tests for important game logic.

At minimum, test:

Match Detection

3 matching tiles → valid match

2 matching tiles → invalid match

Tile Swap

Valid swap → board changes

Invalid swap → board reverts

Scoring

3-match → expected score

4-match → expected score

Combo → multiplier applied

Level Completion

Objective achieved → level completed

Moves exhausted → game over

You may use an appropriate testing framework such as Vitest.

Accessibility

The application should consider accessibility.

At minimum:

- Buttons should have accessible labels
- Interactive elements should be keyboard accessible where practical
- Sufficient visual contrast

- Avoid relying solely on color to communicate state
- Provide meaningful labels for game controls
- Respect reduced-motion preferences where practical

Performance Requirements

The game should remain responsive during:

- Cascading matches
- Multiple simultaneous matches
- Special tile effects
- Rapid user interactions

Avoid unnecessary rendering and state updates.

The application should prevent the user from triggering conflicting game operations while a board animation/update is in progress.

---

npm Publishing Requirement

The StencilJS component library must be packaged and published as an npm package.

The package should:

- Have a meaningful package name
- Include a valid package.json
- Follow semantic versioning
- Include build configuration
- Include usage documentation
- Be installable independently
- Export the required Web Components

Example:

npm install @your-name/match3-ui

The package should be publicly accessible on npm.

Versioning

Follow semantic versioning:

MAJOR.MINOR.PATCH

For example:

1.0.0

1.1.0

1.1.1

The README should explain the current package version.

Deployment

Deploy the SvelteKit application to a publicly accessible platform.

Examples include:

- Vercel
- Netlify
- Cloudflare Pages
- AWS
- Other suitable hosting platforms

The deployed application should be accessible without requiring the evaluator to run the project locally.

Deliverables

The candidate must provide:

1. SvelteKit Application

Complete source code for the game application.

---

2. StencilJS Component Library

Complete source code for the reusable Web Component library.

---

3. Published npm Package

Provide the public npm package link.

Example:

https://www.npmjs.com/package/@your-scope/match3-ui

---

4. Deployed Application

Provide a publicly accessible URL.

Example:

https://your-game.vercel.app

---

5. GitHub Repository

Provide the GitHub repository containing:

/apps

  /game-app

/packages

  /game-ui

README.md

You may also use separate repositories for the application and component library.

README Requirements

The repository must contain a comprehensive README.md.

It should include:

Project Overview

Explain:

- What the game is
- Game mechanics

- Technology stack
- Main features

Setup Instructions

Explain:

git clone <repository>

cd <project>

npm install

Starting Development Server

Clearly explain how to start the SvelteKit application.

Example:

npm run dev

Building the Application

Example:

npm run build

npm run preview

Stencil Library

Explain:

- How the Stencil library is built
- How it is published
- How it is consumed by SvelteKit

---

npm Package

Provide the published npm package link.

GitHub Repository

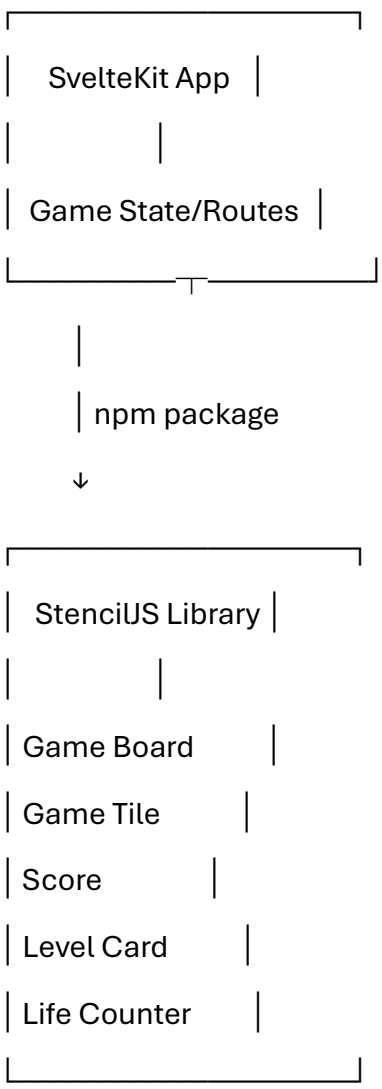
Provide the GitHub repository link.

Deployed Application

Provide the deployed application URL.

Architecture

Include a short explanation or diagram showing:



Assumptions

The README should explicitly document assumptions made during development.

For example:

- No real-money purchases are implemented.
- Player progress is stored locally.
- Levels are provided through static/mock configuration.
- Sound effects may use placeholder/generated assets.
- The game is designed primarily for mobile but supports desktop.
- No authentication is required.
- The game does not require a backend unless the candidate chooses to implement one.

Bonus Features

The following are optional but can demonstrate additional engineering ability.

Power-Ups

Examples:

🔨 Hammer

Remove one tile.

↔ Shuffle

Shuffle the board.

💥 Bomb

Destroy an area.

🌈 Rainbow

Match with any tile.

Daily Challenge

Add a daily puzzle with a unique board/objective.

Leaderboard

Implement a mock leaderboard showing:

1. Alex 25,400

2. Sarah 22,100

3. Player 20,450

Offline Support

Allow previously loaded levels to be played without network connectivity.

PWA

Convert the application into an installable Progressive Web App.

Sound Effects

Add sounds for:

- Matching
- Combos
- Special tiles
- Level completion
- Game over

Dark Mode

Provide light/dark themes.

Advanced Special Tiles

Implement interactions between two special tiles.

For example:

Bomb + Line Blast

↓

Large area explosion

Evaluation Criteria

The assignment will be evaluated based on:

Area	Focus
Game Functionality	Core match-3 mechanics work correctly

Area	Focus
Svelte 5	Appropriate use of Svelte 5 features and architecture
SvelteKit	Routing, application structure and navigation
Game Architecture	Separation of game engine and UI
State Management	Clean and predictable state handling
StencilJS	Quality and reusability of Web Components
Svelte ↔ Stencil Integration	Props, custom events and slots
npm Package	Correctly published and consumable package
UI/UX	Visual quality, responsiveness and usability
Animations	Smooth and meaningful game feedback
Code Quality	Maintainability, readability and structure
Testing	Coverage of important game logic
Performance	Efficient rendering and game execution
Accessibility	Basic accessibility practices
Documentation	Quality of README and setup instructions
Deployment	Working publicly accessible application

Minimum Acceptance Criteria

A submission will be considered complete only if all of the following work:

- Svelte 5 + SvelteKit application
- Original match-3 game
- Level selection
- At least 5 playable levels
- Tile swapping
- Match detection
- Tile removal

- Gravity/refill
- Score system
- Move limitation
- Level objectives
- Win/loss states
- Level progression
- Player progress persistence
- Responsive/mobile-friendly UI
- At least 2 special tile mechanics
- At least 1 obstacle
- At least 5 StenciJS components
- Stencil components published to npm
- SvelteKit consumes the **published npm package**
- Component properties demonstrated
- Custom events demonstrated
- Slots demonstrated
- At least basic game-logic tests
- GitHub repository
- Deployed application
- Complete README

Important

The objective is not to reproduce Candy Crush. The evaluator should be able to see that the candidate has created an **original match-3 game experience** while demonstrating strong frontend engineering, game-state architecture, reusable Web Components, and Svelte 5/SvelteKit integration.